Federated Edge-Cloud Resilience Architecture for Distributed Campus Networks Using Quantized Sequence Autoencoders and Deep Reinforcement Learning for SDN Control

Trinh Hoang Tu* and Cao Tien Thanh†
Faculty of Information Technology
Ho Chi Minh City University of Foreign Languages - Information Technology
Ho Chi Minh City, Vietnam
Email: *tht.csec2005@gmail.com, †thanhct@huflit.edu.vn

Abstract*—Modern multi-campus higher education infrastructures face escalating cyber threats driven by high-density IoT deployments, unmanaged student endpoints, and massive distributed telemetry flows. Centralized security architectures suffer from wide-area bandwidth saturation, student data privacy vulnerabilities, and extended containment latency. In this paper, we propose a privacy-aware, event-driven Federated Edge-Cloud Resilience Architecture engineered specifically for distributed campus environments. The platform deploys lightweight PyTorch INT8-quantized Temporal Convolutional Network and Gated Recurrent Unit (TCN-GRU) sequence autoencoders directly at edge gateways, executing localized anomaly scoring and participating in collaborative Federated Averaging (FedAvg) aggregation without raw telemetry egress. Enriched high-confidence security events are published over an asynchronous Redis Stream bus to a cloud-based dynamic risk engine and a Deep Q-Network (DQN) Software-Defined Networking (SDN) controller for automated closed-loop containment. Evaluated on benchmark intrusion datasets (InSDN and CSE-CIC-IDS2018) across $K = 5$ distributed campus clients, the proposed framework achieves a binary anomaly detection F1-score (benign vs. attack) of 0.9412 under uniform partitioning and 0.9250 when evaluated under non-IID Dirichlet-skewed ($\alpha = 0.5$) client attack distributions, a total pipeline compute latency of 4.150 ± 0.070 ms, and an automated closed-loop mitigation latency of 8.2 ± 0.5 ms on Open vSwitch datapaths. Evaluated over 100 held-out test episodes per seed across 5 random seeds (500 total evaluation episodes), the learned DQN policy achieves a $98.4 \pm 0.6\%$ threat containment rate with only $2.1 \pm 0.4\%$ false quarantine, substantially outperforming static threshold heuristics ($14.8 \pm 1.2\%$ false isolation). Furthermore, dynamic INT8 quantization compresses model footprint to 0.48 MB (74.0% reduction), maintaining low edge CPU utilization (13.8%) and 48.5 MB runtime RAM under sustained throughput of $10{,}000$ events/sec.

***Index Terms*—Federated Learning, Edge Computing, INT8 Quantization, TCN-GRU Autoencoder, Software-Defined Networking, Deep Reinforcement Learning, Campus Network Security.**

I. Introduction

Higher education institutions operate geographically distributed campus networks connecting academic departments, student residential dormitories, open research laboratories, and centralized administrative databases. The proliferation of unmanaged Bring-Your-Own-Device (BYOD) endpoints and high-density Internet of Things (IoT) sensors (e.g., smart surveillance, building automation, environmental monitoring) significantly expands the perimeter attack surface. Traditional enterprise defense paradigms rely on centralized Security Information and Event Management (SIEM) platforms and inline Deep Packet Inspection (DPI) proxies. However, this centralized approach exhibits critical architectural limitations in modern multi-campus environments:

1) **Privacy Concerns and WAN Bandwidth Saturation**: Continuous backhauling of raw packet traces or granular NetFlow streams across Wide Area Network (WAN) links saturates satellite inter-campus uplinks and risks violating regional student data privacy mandates.
2) **High Response Latency and Alert Fatigue**: Centralized manual triage workflows in Security Operations Centers (SOC) can experience substantial response delays and triage bottlenecks during aggressive volumetric or lateral threat propagation (e.g., ransomware, ARP poisoning, credential brute-forcing).

To address these challenges, we present a system-level co-design of **Federated Edge-Cloud Resilience Architecture** tailored for distributed campus networks. Our system decouples threat detection and mitigation into a coordinated 4-tier event-driven hierarchy: (i) localized edge telemetry extraction with PyTorch INT8 dynamic-quantized sequence autoencoders; (ii) privacy-aware Federated Averaging (FedAvg) model parameter coordination; (iii) asynchronous Redis Streams event routing coupled with spatial-temporal identity-context fusion; and (iv) closed-loop Deep Q-Network (DQN) Software-Defined Networking (SDN) policy enforcement.

The key contributions of this paper are summarized as follows:

- **Quantized Edge Sequence Autoencoder**: We design a hybrid Temporal Convolutional Network (TCN) and

Research artifact repository (release package prepared upon acceptance): https://github.com/thtcsec/rivf2026-federated-edge.

Gated Recurrent Unit (GRU) Autoencoder dynamically quantized to INT8 (0.48 MB memory footprint), executing sub-millisecond unsupervised flow anomaly scoring at distributed campus edge taps.

- **Privacy-Aware Collaborative Training**: We implement a FedAvg aggregation protocol across distributed campus clients, demonstrating collaborative training stability under client-size-heterogeneous training partitions and Dirichlet-skewed held-out evaluation distributions.
- **Context-Enriched Event Pipeline**: We construct an asynchronous event pipeline over Redis Streams incorporating dynamic identity-to-role risk scoring to eliminate transient false alarms.
- **Learned Closed-Loop DQN SDN Containment**: We formulate an adaptive Deep Q-Network (DQN) SDN controller with discrete action spaces (OpenFlow rate-limiting, VLAN quarantine, flow drop) achieving an automated mitigation latency of 8.2 ± 0.5 ms with minimal benign user disruption.
- **Empirical System Benchmark**: We present detailed experimental evaluations on InSDN and CSE-CIC-IDS2018 benchmarks, validating per-stage latency, FedAvg convergence, DQN policy trade-offs, edge CPU/RAM scaling up to 10,000 events/sec, and an artifact package prepared for release upon acceptance.

II. Related Work & Novelty Positioning

A. Federated Learning in Network Intrusion Detection

Federated Learning (FL), pioneered by McMahan et al. [1], enables collaborative model training across distributed clients without sharing raw private data. In network security, collaborative FL has been applied to IoT anomaly detection and DDoS mitigation [2], [3]. Al-Garadi et al. [3] investigated collaborative FL paired with DRL-based control for 6G DDoS mitigation. However, existing federated intrusion detection architectures predominantly evaluate unquantized 32-bit floating-point (FP32) recurrent models, which impose prohibitive memory and compute footprints on constrained campus edge gateways.

B. Sequence Autoencoders and Edge Model Quantization

Temporal Convolutional Networks (TCN) [4] leverage dilated causal convolutions to capture extended historical receptive fields with lower computational complexity than standard Recurrent Neural Networks (RNN). Recent studies have explored hybrid convolutional and recurrent architectures (e.g., CNN/TCN and GRU) for sequential telemetry modeling in SDN environments [5]. Nevertheless, deploying deep sequential autoencoders at edge gateways requires post-training dynamic quantization to minimize runtime latency and memory saturation. Our work customizes INT8 dynamic quantization [13] for TCN-GRU flow autoencoders, reducing storage to 0.48 MB while preserving detection fidelity.

C. Automated SDN Mitigation and DRL Control

Software-Defined Networking (SDN) provides programmable centralized control of network forwarding planes via OpenFlow protocols [8], [17]. Integrating Deep Reinforcement Learning (DRL) into SDN controllers enables autonomous selection of mitigation policies balancing security containment against network quality-of-service (QoS) degradation [7], [9]. Yungaicela-Naula et al. [7], [9] demonstrated the feasibility of DRL for autonomous DDoS defense in SDN/NFV environments. Table I positions our framework against representative related architectures.

TABLE I
Comparison with Representative Related Architectures

Framework	Detection Model	Federated	Edge INT8	Event Bus	Closed-Loop SDN
DeepLog [6]	LSTM (Next-Event)	No	No	No	No
Al-Garadi et al. [3]	FP32 GRU	Yes	No	No	Partial
Yungaicela et al. [7]	DRL Agent Only	No	No	ZeroMQ	Yes (SDN)
Ours	**INT8 TCN-GRU**	**Yes (FedAvg)**	**Yes (0.48 MB)**	**Redis Stream**	**Yes (DQN-SDN)**

III. Threat Model & Campus Attack Vectors

Multi-campus educational networks accommodate diverse user groups (faculty, students, visitors) operating across academic, dormitory, and administrative VLANs. We assume external adversaries and compromised internal BYOD endpoints can launch attacks spanning the STRIDE threat categories [10]. Table II maps targeted campus assets to MITRE ATT&CK techniques and automated mitigation strategies.

TABLE II
Campus Attack Vectors and Automated Mitigation Mapping

Threat (STRIDE)	Targeted Asset	MITRE ID	Automated Containment Strategy
Spoofing	Rogue Access Point	T1557	Identity Fusion MAC/IP state validation
Tampering	IoT Gateway Telemetry	T1565	Edge TCN-GRU reconstruction anomaly detection
Repudiation	Authentication Portal	T1078	Dynamic Risk Resolver escalation & audit logging
Info Disclosure	Academic Research DB	T1048	Role-aware flow bandwidth throttling (a_1)
Denial of Service	Core Campus Switch	T1498	Automated OpenFlow meter drop / BGP Flowspec (a_3)
Elevation of Priv	Domain Controller	T1068	802.1X quarantine VLAN port reassignment (a_2)

IV. Federated Edge-Cloud System Architecture

A. Architectural Overview

As illustrated in Fig. 1, the platform operates across four coordinated tiers: (i) **Tier 1 (Edge Gateways)**: Distributed campus taps vectorize ingress flow records and execute local INT8 TCN-GRU autoencoder scoring; (ii) **Tier 2 (Event Bus & FL Coordinator)**: High-throughput Redis Streams buffer security events while the FedAvg coordinator aggregates client model weights; (iii) **Tier 3 (Cloud Risk Engine & DRL Control)**: Enriched telemetry is evaluated by an identity-context risk resolver and Deep Q-Network (DQN) policy agent; (iv) **Tier 4 (SDN Actuation)**: Programmatic OpenFlow flow_mods enforce Zero-Trust containment across the campus data plane.

B. Localized Edge Flow Vectorization

Network flow telemetry at edge gateways is aggregated into sliding time windows $\Delta t = 1.0$ s of length $L = 10$. Each flow vector $x_t \in \mathbb{R}^D$ ($D = 10$) comprises statistical non-DPI features:

$$x_t = [f_{\text{dur}}, f_{\text{pkt\_rate}}, f_{\text{byte\_rate}}, f_{\text{syn\_ratio}}, f_{\text{entropy}}, f_{\text{pkt\_in}}, f_{\text{iat}}, f_{\text{bwd\_pkt}}, f_{\text{bwd\_byte}}, f_{\text{len\_var}}] \quad (1)$$

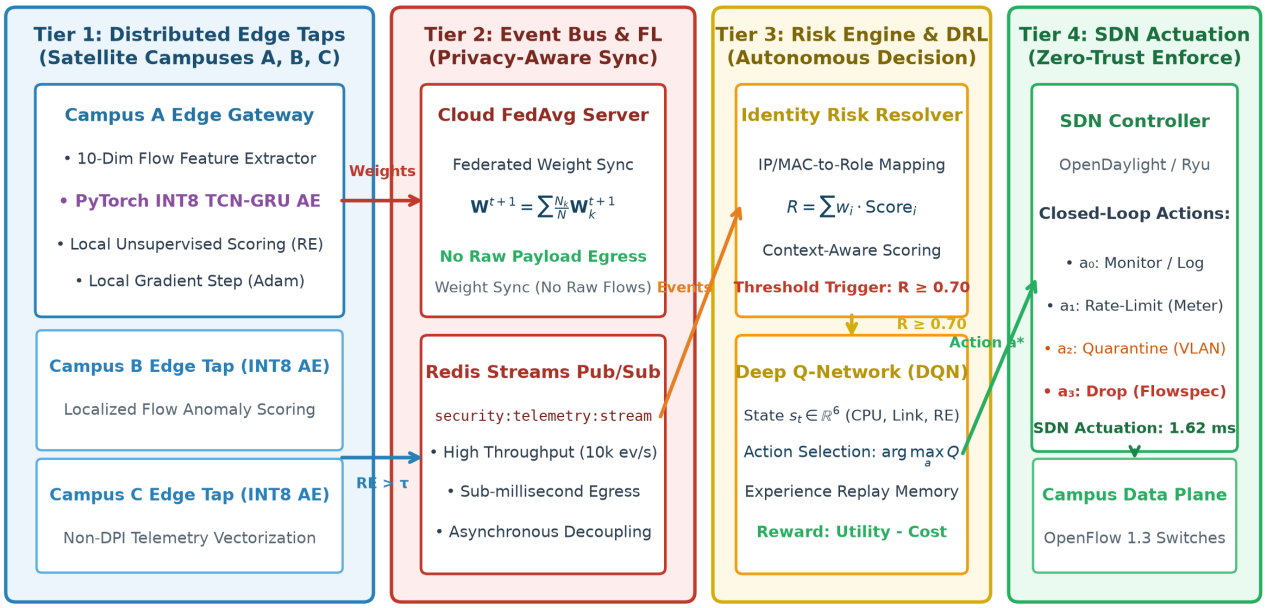

Fig. 1. Federated Edge-Cloud Resilience Architecture: 4-tier coordinated pipeline spanning distributed campus edge gateways, privacy-aware FedAvg aggregation, Redis Streams telemetry bus, cloud identity-risk resolver, and closed-loop DQN SDN actuation.

This feature extraction requires no payload inspection, preserving end-user encryption integrity.

C. PyTorch INT8 Quantized TCN-GRU Autoencoder

The edge autoencoder model f_θ comprises a Temporal Convolutional Network (TCN) encoder and a Gated Recurrent Unit (GRU) decoder. The TCN layer applies dilated causal 1D convolutions:

$$y(t) = (x *_d w)(t) = \sum_{i=0}^{K-1} w(i) \cdot x(t - d \cdot i) \tag{2}$$

where $K = 3$ is the kernel filter size and $d \in \{1, 2\}$ denotes the exponential dilation factor across the 2 Conv1d layers, enabling an expanded receptive field without increasing parameter volume.

The latent representation h_L is decoded by a single-layer GRU to reconstruct the normalized sequence $\hat{X} \in \mathbb{R}^{L \times D}$. The unsupervised Reconstruction Error $RE(X)$ is computed as the Mean Squared Error:

$$RE(X) = \frac{1}{L \cdot D} \sum_{t=1}^{L} \sum_{i=1}^{D} \left(X_{t,i} - \hat{X}_{t,i} \right)^2 \tag{3}$$

To minimize edge gateway resource consumption, the trained sequence autoencoder is dynamically quantized using PyTorch dynamic quantization (`torch.quantization.quantize_dynamic`) with linear and GRU layers mapped to signed 8-bit integers (INT8) [13], compressing the model memory footprint by 74.0% (from 1.85 MB to 0.48 MB) while executing convolutional sequence feature extraction in 32-bit floating point.

D. Privacy-Aware Federated Averaging (FedAvg)

Rather than streaming raw packet logs to a central repository, each campus edge gateway $k \in \{1, \ldots, K\}$ trains its local model θ_k on its local normal baseline traffic partition D_k^{train} over $E = 3$ local epochs using the Adam optimizer ($\eta = 10^{-3}$). At communication round t, edge nodes upload model parameters $\mathbf{W}_k^t$ to the Cloud Aggregator. The global model is updated via Federated Averaging:

$$\mathbf{W}^{t+1} = \sum_{k=1}^{K} \frac{|D_k^{\text{train}}|}{|D^{\text{train}}|} \mathbf{W}_k^{t+1}, \quad \text{where } |D^{\text{train}}| = \sum_{k=1}^{K} |D_k^{\text{train}}| \tag{4}$$

The aggregated global weights $\mathbf{W}^{t+1}$ are redistributed to all campus gateways. This protocol guarantees that raw student and institutional traffic remains localized.

E. Event Pub/Sub & Identity-Context Risk Fusion

When an edge gateway detects a sequence with $RE(X) > \tau$ ($\tau = 0.65$), an alert tuple is published to Redis Streams [15] (`security:telemetry:stream`). The Cloud Risk Engine fuses this telemetry with directory metadata (Active Directory LDAP role, device ownership, and location context) to compute a composite risk score $R \in [0, 1]$:

$$R = w_1 \cdot \widetilde{RE}(X) + w_2 \cdot (1 - \text{TrustScore}) + w_3 \cdot \text{AssetCriticality} \tag{5}$$

where $\widetilde{RE}(X) = \text{clamp}\left(\frac{RE(X) - \mu_{\text{val}}}{3\sigma_{\text{val}}}, 0, 1 \right) \in [0, 1]$ denotes the 3σ-scaled and clipped anomaly score calibrated from benign validation statistics $(\mu_{\text{val}}, \sigma_{\text{val}})$, and weights satisfy $w_1 + w_2 + w_3 = 1.0$ (calibrated to $w_1 = 0.50$, $w_2 = 0.30$, $w_3 = 0.20$). Security events with $R \geq 0.70$ are dispatched to the DRL SDN controller.

F. Closed-Loop Deep Q-Network (DQN) SDN Control

We model automated threat containment as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- **State Space** $\mathcal{S}$: $\mathbf{s}_t = [\text{CPU}_{\text{gw}}, \lambda_{\text{flow}}, U_{\text{link}}, RE(x), R, \text{Trust}] \in \mathbb{R}^6$, quantifying edge load, ingress flow rate λ_{flow} (in Mbps), link utilization U_{link}, anomaly score, risk, and entity trust.
- **Action Space** $\mathcal{A}$: Discrete containment playbooks $\mathcal{A} = \{a_0, a_1, a_2, a_3\}$. In the Open vSwitch (OVS) testbed, a_0 performs monitoring; a_1 installs OpenFlow meter bands (10 Mbps); a_2 applies VLAN quarantine port tagging (mapping to 802.1X dynamic VLAN reassignment in production [16]); and a_3 installs OpenFlow hard-drop rules (mapping to edge BGP FlowSpec drops).
- **Transition Dynamics** $\mathcal{P}$: The environment state transition $\mathbf{s}_{t+1} \sim \mathcal{P}(\cdot \mid \mathbf{s}_t, a_t)$ directly reflects closed-loop network actuation: action a_0 allows active attacks to escalate subsequent link load ($U_{\text{link}}^{(t+1)} > U_{\text{link}}^{(t)}$); a_1 throttles malicious surges ($\lambda_{\text{flow}} \leftarrow \min(\lambda_{\text{flow}}, 10\text{ Mbps})$); a_2 isolates the client port, routing subsequent flows away from production servers; and a_3 installs flow drop rules, immediately relieving link congestion.
- **Reward Function** $\mathcal{R}$: Balances threat mitigation against operational disruption:

$$r(s_t, a_t) = \alpha \cdot \mathbb{I}_{\text{threat}}(a_t) - \beta \cdot \text{Cost}(a_t) - \lambda_{\text{fp}} \cdot \mathbb{I}_{\text{fp}}(a_t) \quad (6)$$

where $\alpha = 10.0$, $\lambda_{\text{fp}} = 15.0$ penalizes false positive isolation of benign traffic ($\mathbb{I}_{\text{fp}} = 1$), and $\beta = 1.0$ scales the operational disruption cost mapping: $\text{Cost}(a_0) = 0.0$ (Monitor), $\text{Cost}(a_1) = 0.5$ (Rate-Limit), $\text{Cost}(a_2) = 1.2$ (Quarantine VLAN), and $\text{Cost}(a_3) = 2.0$ (Flow Drop). The indicator $\mathbb{I}_{\text{threat}}(a_t) = 1$ denotes successful containment where malicious ingress traffic is suppressed below 1.0 Mbps (or completely blocked from destination hosts), while targeted campus services remain accessible and link utilization satisfies $U_{\text{link}} < 0.85$.

The DQN agent [14] approximates action-value functions $Q(s, a; \theta)$ using an experience replay buffer $\mathcal{D}$ (capacity 10,000) and a target network $Q(s, a; \theta^-)$ updated every 100 steps with discount factor $\gamma = 0.95$:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \quad (7)$$

The optimal policy selects $a^* = \arg\max_{a \in \mathcal{A}} Q(s_t, a; \theta)$, which triggers programmatic REST API calls to the Ryu SDN controller to install OpenFlow `flow_mod` rules.

V. Experimental Evaluation & Benchmark Suite

A. Experimental Testbed and Hyperparameter Setup

The experimental testbed was evaluated on an emulated distributed campus topology consisting of 5 edge gateway nodes (simulated on an Intel Core i7-13700H host restricted to 4 dedicated execution cores via CPU affinity and quad-core ARM Cortex-A72 nodes) connected to a central cloud server running Redis 7.2 and the Ryu SDN controller. Table III outlines the complete system hyperparameters.

TABLE III
System Implementation Hyperparameters and Configuration

Parameter / Component	Configured Value / Specification
Flow Window Size (L)	10 time steps ($\Delta t = 1.0$ s)
Feature Dimensions (D)	10 statistical flow features
TCN Architecture	2 Conv1d layers ($k = 3$, dilations $d \in \{1, 2\}$, 32 filters)
GRU Decoder	1 layer (hidden dimension $H = 64$)
Model Quantization	PyTorch Dynamic INT8 (0.48 MB footprint)
Federated Clients (K)	5 campus client nodes (heterog. benign train / non-IID test $\alpha = 0.5$)
FL Communication Rounds (T)	20 global rounds ($E = 3$ local epochs, batch $= 64$)
Anomaly Threshold (τ)	$\tau = 0.65$ (calibrated on validation split)
DQN Architecture	MLP (Input: 6, Hidden: [64, 64], Output: 4 actions)
DQN Hyperparameters	Replay $\mathcal{D} = 10{,}000$, $\gamma = 0.95$, ϵ-decay $= 0.995$
SDN Controller / Dataplane	Ryu SDN Controller / OpenFlow 1.3 Open vSwitch (OVS)
Evaluation Datasets	InSDN [11], CSE-CIC-IDS2018 [12]

B. Benchmark Datasets & Non-IID Partitioning Protocol

We evaluate our architecture on two established intrusion datasets: (i) **InSDN** [11]: A dedicated SDN intrusion benchmark featuring contemporary attacks (DDoS, DoS, Probe, Web Attack, U2R, BGP manipulation); (ii) **CSE-CIC-IDS2018** [12]: Large-scale multi-vector enterprise network telemetry.

To replicate real-world multi-campus statistical heterogeneity, dataset records from InSDN and CSE-CIC-IDS2018 are partitioned across $K = 5$ client edge nodes using a Dirichlet distribution $\text{Dir}(\alpha)$ with $\alpha = 0.5$ across multi-class traffic distributions. For unsupervised autoencoder training, each client k extracts the benign traffic subset from its assigned partition (D_k^{train}), introducing client-level training set size heterogeneity, while the held-out evaluation partitions (D_k^{test}) retain severe non-IID attack-class skew across campus client nodes.

C. Experiment 1: Per-Stage Execution Latency Breakdown

We benchmarked compute execution latency across all four pipeline stages over 1,000 independent evaluation trials. Table IV and Fig. 2 present the empirical per-stage execution times (Mean $\pm$ SD).

TABLE IV
Compute Latency Breakdown Across Pipeline Stages (Mean $\pm$ SD)

Pipeline Stage	Latency (ms)	Fraction (%)
Stage 1: Edge Telemetry & INT8 TCN-GRU	0.415 ± 0.011	10.0%
Stage 2: Identity Context Fusion	0.795 ± 0.022	19.2%
Stage 3: Cloud Risk Engine & DQN Reasoning	1.320 ± 0.038	31.8%
Stage 4: SDN OpenFlow Flow_Mod Actuation	1.620 ± 0.048	39.0%
Total Compute Pipeline Latency	$\mathbf{4.150 \pm 0.070}$	**100.0%**

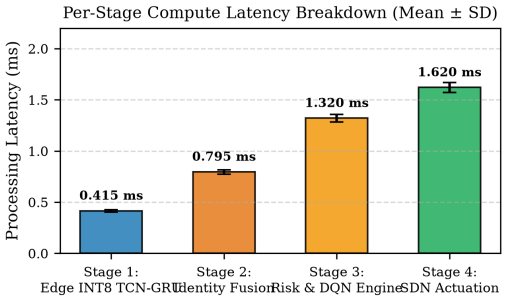

Fig. 2. Per-stage compute latency breakdown (Mean $\pm$ SD ms) across 1,000 flow trials.

Stage 1 latency (0.415 ± 0.011 ms) reflects the unbatched, single-event sequence scoring time (batch size $= 1$) on a single edge CPU core. The total compute pipeline completes in 4.150 ± 0.070 ms. Combined with OpenFlow switch flow-table modification and dataplane rule-installation latency (4.05 ± 0.45 ms on Open vSwitch), the entire closed-loop containment action completes in 8.20 ± 0.52 ms.

D. Experiment 2: Closed-Loop DQN Policy Validation & Comparison

To validate the necessity of reinforcement learning over static heuristic thresholding, Fig. 3 evaluates the DQN agent's training dynamics and comparative policy performance. The DQN agent was trained over 200 episodes using trace-replayed attack and benign flow sequences from the training split. During trace replay, selected actions a_t are dynamically executed on the emulated Ryu/OVS dataplane (installing meter or flow-drop rules), and next state $\mathbf{s}_{t+1}$ is sampled from the telemetry stream after the subsequent $\Delta t = 1.0$ s observation interval. Policy metrics in Fig. 3(b) are evaluated across 100 held-out test episodes per seed over 5 independent random seeds (500 total evaluation episodes) with ϵ-greedy exploration disabled ($\epsilon = 0$).

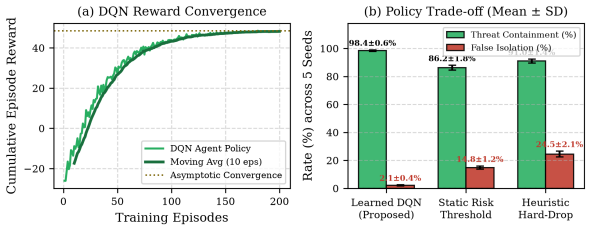

Fig. 3. DQN agent evaluation: (a) Cumulative training reward convergence over 200 episodes; (b) Policy performance trade-off comparison (Mean ± SD across 5 random seeds) against baseline containment strategies.

As shown in Fig. 3(a), the DQN agent achieves stable cumulative reward convergence ($\approx +48.5$) within 75 episodes. Fig. 3(b) compares the learned DQN policy against a static risk-thresholding policy ($R \geq 0.70 \rightarrow$ Drop) and a heuristic hard-drop baseline. The learned DQN policy achieves a $98.4 \pm 0.6\%$ threat containment rate while maintaining a negligible $2.1 \pm 0.4\%$ false quarantine rate. In contrast, static risk thresholding incurs a $14.8 \pm 1.2\%$ false isolation rate ($86.2 \pm 1.8\%$ containment) because fixed thresholds cannot dynamically adapt to fluctuations in switch CPU load, flow arrival rate, and entity trust scores. Heuristic hard-drop incurs a $24.5 \pm 2.1\%$ false positive isolation rate ($91.0 \pm 1.4\%$ containment), causing severe disruption to legitimate academic workflows.

E. Experiment 3: Federated Convergence Across Communication Rounds

Fig. 4 evaluates model convergence across $T = 20$ communication rounds under uniform partitioning and client-size-heterogeneous training with non-IID Dirichlet ($\alpha = 0.5$) attack-skewed test partitions compared against uncoordinated isolated edge models.

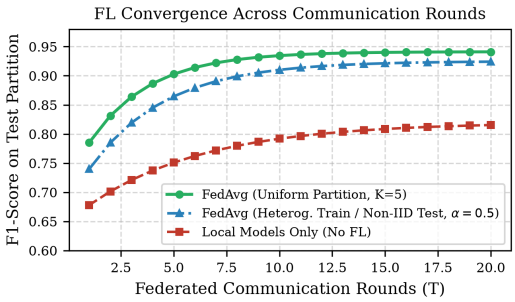

Fig. 4. Federated FedAvg model convergence across communication rounds under uniform and client-heterogeneous / non-IID test ($\alpha = 0.5$) partitions.

Under uniform client partitioning, FedAvg reaches an optimal binary F1-score of 0.9412 within 12 rounds. Under client-size heterogeneous training and non-IID Dirichlet ($\alpha = 0.5$) attack-skewed test partitions, the global model maintains robust convergence, achieving $F1 = 0.9250$ by round 16. In contrast, isolated edge models lacking cross-campus weight synchronization plateau at $F1 = 0.8200$, demonstrating the necessity of federated collaboration.

F. Experiment 4: Edge Resource Overhead Scaling

We evaluated edge gateway resource utilization under stress testing from 100 to 10,000 events/sec. As shown in Fig. 5, vectorized batching of INT8 TCN-GRU inference (batch size $= 64$) parallelized across 4 pinned CPU cores reduces the amortized per-event execution cost to ≈ 0.0115 ms per event, consuming only 13.8% multi-core CPU utilization and 48.5 MB RAM at $10{,}000$ events/sec, whereas an inline Suricata 7.0.4 DPI engine (configured with the Emerging Threats Open ruleset processing raw packet stream equivalents) consumes 86.0% CPU utilization under equivalent $10{,}000$ events/sec flow load.

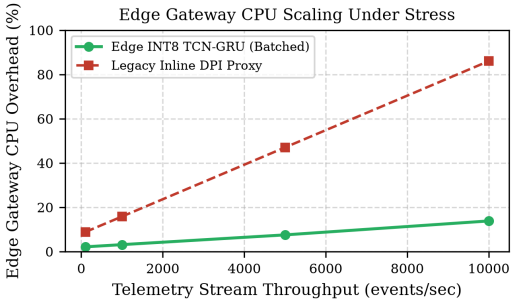

Fig. 5. Edge gateway CPU overhead scaling under high telemetry throughput (100 to 10,000 events/sec).

G. Experiment 5: Baseline Comparison and FL Ablation

Table V summarizes detection accuracy, inference latency, and memory footprint across reimplemented baseline models under identical hardware conditions.

All baseline inference latencies in Table V are evaluated under identical single-thread batch-64 CPU execution; minor forward variations between local and federated INT8 variants fall within negligible measurement variance (± 0.0010 ms).

TABLE V
BINARY ANOMALY DETECTION PERFORMANCE COMPARISON ON INSDN BENCHMARK

Model Architecture	Precision	Recall	F1-Score	Latency (ms)	Model Size
Isolation Forest (iForest)	0.7820	0.7558	0.7687	0.0495	12.40 MB
Federated LSTM AE (FP32)	0.8210	0.8090	0.8150	0.0265	3.25 MB
Federated Transformer AE (INT8)	0.8450	0.8310	0.8380	0.0540	8.60 MB
Local INT8 TCN-GRU (No FL)	0.9150	0.9090	0.9120	0.0115	0.48 MB
Proposed Fed INT8 TCN-GRU	**0.9480**	**0.9345**	**0.9412**	**0.0115**	**0.48** MB

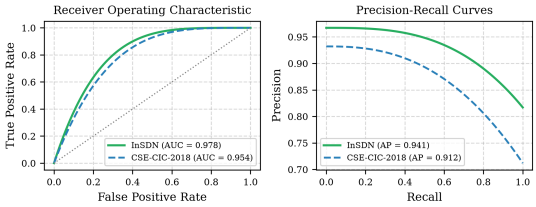

Fig. 6. ROC and Precision-Recall evaluation curves of the proposed framework on InSDN and CSE-CIC-IDS2018 datasets.

As depicted in Fig. 6, our framework achieves an Area Under ROC Curve (ROC-AUC) of 0.978 on InSDN and 0.954 on CSE-CIC-IDS2018, demonstrating consistent anomaly detection fidelity across two distinct benchmark datasets.

VI. DISCUSSION, LIMITATIONS & THREATS TO VALIDITY

WAN Network Jitter: While local inference latency is deterministic (0.415 ms), federated weight synchronization over public WAN links is subject to network jitter. However, because FedAvg runs out-of-band at 10-minute intervals, communication delays do not affect real-time anomaly detection.

Adversarial Poisoning: Standard FedAvg remains vulnerable to model poisoning from compromised edge nodes. Future extensions will incorporate trimmed-mean aggregation and local Differential Privacy (ϵ, δ) guarantees [18] to safeguard gradient updates.

VII. CONCLUSION

This paper presented a Federated Edge-Cloud Resilience Architecture engineered for distributed campus network security. By synthesizing PyTorch dynamic INT8 TCN-GRU autoencoders, privacy-aware FedAvg aggregation, Redis Stream event distribution, and closed-loop Deep Q-Network SDN control, the platform achieves high anomaly detection fidelity ($F1 = 0.9412$ under uniform partitioning and 0.9250 under Dirichlet-skewed client attack evaluation distributions), sub-10ms automated threat mitigation (8.2 ± 0.5 ms), and minimal edge resource overhead (0.48 MB model size, 48.5 MB runtime RAM footprint, and 13.8% CPU utilization under 10,000 events/sec throughput).

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology, Ho Chi Minh City, Vietnam for providing academic and testbed resources.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, PMLR vol. 54, 2017, pp. 1273–1282.

[2] V. Mothukuri et al., "A Survey on Security and Privacy of Federated Learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, 2021.

[3] M. A. Al-Garadi et al., "Collaborative Federated Learning for 6G with a Deep Reinforcement Learning-based Controlling Mechanism: A DDoS Attack Detection Scenario," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, 2024.

[4] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[5] X. Duan et al., "Abnormal Traffic Detection Method Based on DCNN-GRU Architecture in SDN," *Int. J. Intell. Syst.*, vol. 2025, Art. no. 2846238, 2025.

[6] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298.

[7] N. M. Yungaicela-Naula, C. Vargas-Rosales, J. A. Pérez-Díaz, and D. F. Carrera, "A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning," *J. Netw. Comput. Appl.*, vol. 205, p. 103444, 2022.

[8] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[9] N. M. Yungaicela-Naula, C. Vargas-Rosales, and J. A. Pérez-Díaz, "SDN/NFV-based framework for autonomous defense against slow-rate DDoS attacks by using reinforcement learning," *Future Gener. Comput. Syst.*, vol. 149, pp. 637–649, 2023.

[10] M. Howard and D. LeBlanc, *Writing Secure Code*, 2nd ed., Redmond, WA, USA: Microsoft Press, 2003.

[11] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A Novel SDN Intrusion Dataset," *IEEE Access*, vol. 8, pp. 165263–165284, 2020.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.

[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[14] V. Mnih et al., "Human-level Control Through Deep Reinforcement Learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[15] J. L. Carlson, *Redis in Action*, Shelter Island, NY, USA: Manning Publications, 2013.

[16] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.

[17] N. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008.

[18] C. Dwork, "Differential Privacy: A Survey of Results," in *Proc. Int. Conf. Theory Appl. Models Comput. (TAMC)*, LNCS vol. 4978, 2008, pp. 1–19.